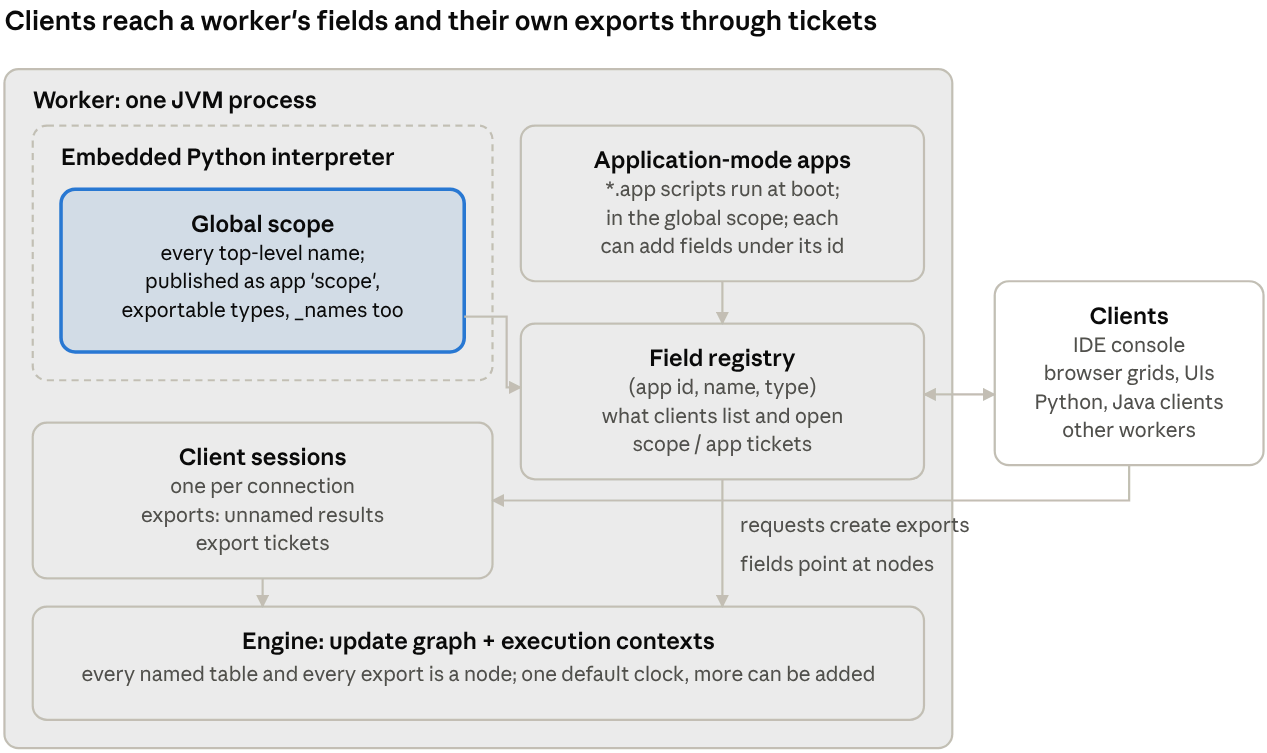
Deephaven Concepts: Scopes, Fields, Tickets and Barrage

Oct 20, 2026 · @Taylor

A Deephaven app is everything a worker built and kept alive: its interface is its named fields, and its internals are the update graph, listeners and threads behind them. Clients reach fields through tickets, and ticking data travels over Barrage.

At a glance



anatomy of one worker · scope, apps, sessions, engine

Named objects reach clients only through the field registry; everything a client derives lives as an unnamed export in its session, and both end up as nodes in the same update graph.

Glossary

The terms nest: a worker holds one Python namespace, the server publishes parts of it as fields, and clients reach fields through tickets.

Term	What it is	Why it matters
Worker	One Deephaven server process: a JVM with an embedded Python interpreter. On Enterprise, each PQ runs in its own worker	The unit that owns a scope, its update graphs and its threads. One server per process (**Verified**)
Global scope (console scope)	The worker's single Python namespace, the `__main__` module's globals. Console commands, `runCode` / `run_script` calls and application-mode scripts all run in it. With the pip server, it is the launching script's own `__main__`	There is one, not several. Every console and client shares it, so names can collide, and a client can read and modify whatever the host process keeps there. **Verified**
`runCode` / `run_script`	The same request under two names (JS API / Python client): run this code in the worker's global scope	No new local scope: a name defined by one call is visible to the next. **Verified**
Script session	The server object that owns the interpreter and its global scope, and runs commands one at a time	What "attaching a console" connects you to; app-mode scripts run through it too
Field	A named, typed object the server publishes to clients: (application id, name, type). Any registered object type qualifies: tables, figures, deephaven.ui elements, partitioned and hierarchical tables, plugin objects	The real list of what is exposed. An address book, not a permission boundary

Term	What it is	Why it matters
The scope application	The global scope presented as fields, under application id `scope`. Every exportable object bound to a global name appears, underscore names included	Publishing is opt-out: keep a table private by not binding it to a global name. **Verified**
Application mode	Startup scripts named by `*.app` files. Each application has an id and can publish fields with `get_app_state()["name"] = obj`	In Community 42.6 the scripts run in the global scope, so plain top-level names also become `scope` fields; `global` is not needed. **Verified**
Application	A named container of fields: `scope`, one per `*.app` file, plus built-ins. Many per worker; two apps may use the same field name	A namespace, not an isolation boundary: all apps share one JVM and interpreter, and by default one update graph (more can be created). **Verified**
Export	An unnamed object a client session created or opened, such as a sorted grid view	A real, ticking graph node that appears in no scope, unless the client binds it to a name
Ticket	The handle a client uses to refer to a field or an export	Every client request is phrased in tickets (see Tickets below)
Query scope	The names that query-language formulas can refer to, such as `MULT` in `"Px * MULT"`, or a Python function. In Python they resolve from where the table operation was written	How formulas call your Python functions
Execution context	A Java bundle that every table operation runs under: query scope, query library, formula compiler, update graph, auth context. Contexts are **per thread**	The script session and the thread that started the server hold the real one. On any other thread, `get_exec_ctx()` returns a static-only context and ticking tables fail, so capture the real one there and pass it in. **Verified**

Term	What it is	Why it matters
Update graph	A clock plus a dependency graph of table operations and listeners	One default periodic graph (about 1 s); more can be created, including an event-driven one that advances only when your code calls `requestRefresh()`. **Verified**
Barrage	Deephaven's protocol for streaming ticking tables: a snapshot, then incremental updates	How grids, Python clients and other workers receive live data

The model, tested

A worker has one Python namespace, and everything named in it is published: console commands, `run_script` calls and application-mode scripts all share it, and each application adds its own fields on top.

In one paragraph: a worker is one JVM with one embedded Python interpreter and one default update graph (more can be created, including an event-driven one). Its script session owns a single global scope, and every exportable object bound to a name there is a field of the `scope` application. Application-mode apps can publish further fields under their own ids. Together these form the field registry clients see. Each client session holds export tickets for objects it fetched or created; these can branch from any node, add new roots, or be promoted into the scope by binding them. Every table, named or not, is a node in an update graph: the default one, unless it was built under an execution context bound to another.

We checked each claim on Deephaven Community 42.6, running natively on Windows, mostly with a two-app probe worker (`tests/probe_concepts.py`).

Claim tested	Result
A name defined by one `run_script` call is visible to the next	Yes: both calls ran in the global scope

Claim tested	Result
App-mode scripts run in their own namespace and need `global`	**No.** Inside the app script, `globals() is __main__.__dict__` was True, and an undeclared top-level table appeared as a `scope` field
With the embedded pip server, the console scope is the launcher's `__main__`	Yes: the launcher's own `main()` appeared in the scope, and a client's `run_script` read **and modified** a dict the host kept in its module globals
Underscore-prefixed names stay private	**No.** `_probe_underscore` and `_rs_underscore` were both published as `scope` fields
A worker can hold several applications	Yes: the registry listed `scope`, `probe.scoping` and `probe.second`. A built-in app with no exports does not appear
Two apps can each have a field with the same name	Yes: `probe_field` resolved to different tables under each app id
`bind_table` promotes a client export to a field	Yes: the name was still a field, with its 3 rows, after the binding session closed
A shared ticket outlives its publisher	**No.** Fetching it failed once the publishing session closed, and so did a copy another session had already fetched
URI forms	`dh+plain://host:port/scope/<name>` and `dh+plain://host:port/app/<id>/field/<name>` resolved; `local:///...` found no resolver. A resolved URI appeared in the worker's graph as a Barrage source node, even pointing at itself
`get_exec_ctx()` works on any thread	**No.** On the host's main thread, with the server started on another thread, it returned a static-only context; the context captured on the server's thread worked
A worker has exactly one clock	**No.** An event-driven graph built from Python held three pushes until one `requestRefresh()` delivered them as a single step

Five earlier claims were wrong and are corrected in these tables. App-mode scripts do not need `global` in Community Python; an underscore does not keep a name out of the fields; a shared ticket dies with its publishing session; get_exec_ctx() returns a real

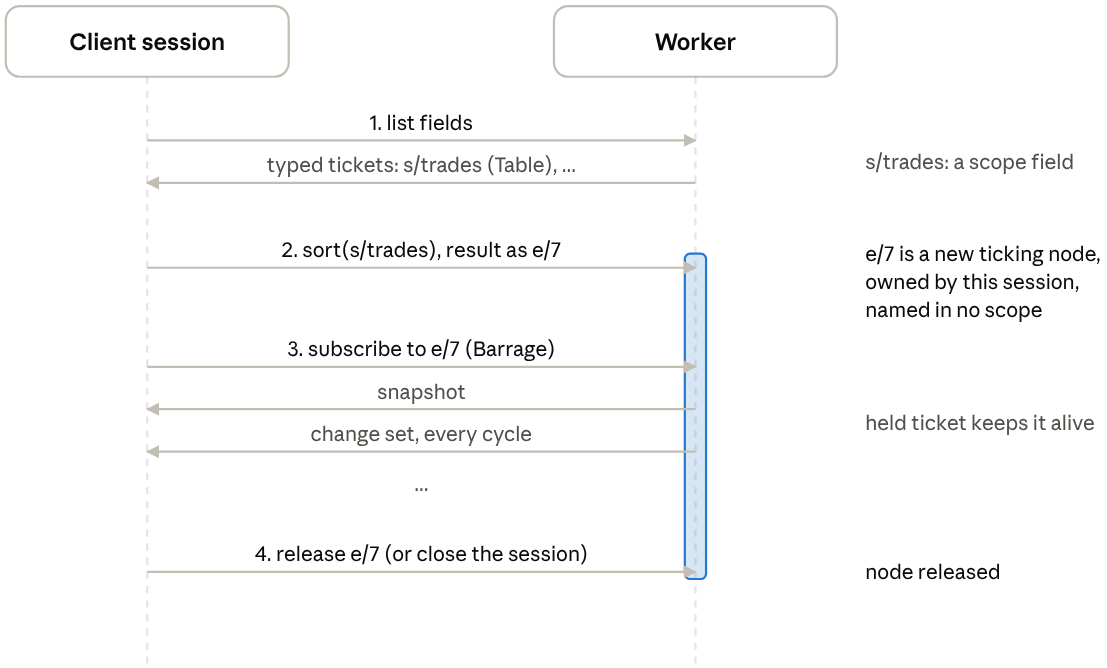
context only on a thread that already holds one; and a worker has one clock by default but can run more. The Deephaven docs advise declaring `global` in app scripts; that may still matter for other script types or versions.

Tickets

A ticket is an opaque handle naming a server-side object; every client request takes tickets in and hands new tickets back. The term comes from Apache Arrow Flight, the transport Deephaven's client API is built on, where you fetch a stream by presenting its ticket.

A typical request reads: take the object named by ticket X, apply this operation, put the result under new ticket Y, now stream me Y.

An export ticket lives from the request that creates it until release



ticket lifecycle · one client sorting a grid

The shaded bar is the export's lifetime: it ticks in the worker's update graph for as long as the session holds its export ticket (export #7 here; the ticket itself is the byte e plus a 4-byte number).

Kind	Bytes	Names	Lifetime
Scope ticket	`s/<name>`	A field in the global scope, such as `s/trades`	As long as the name is bound
Application ticket	`a/<app id>/f/<field>`	A field published by an application	As long as the app holds it
Export ticket	`e` + a 4-byte number (5 bytes)	Something this client session created or fetched	Until the client releases it or its session ends; it keeps the object alive while held
Shared ticket	`h` + a random 16-byte id	An export one session publishes for other sessions	Only while the **publishing** session holds it: it dies when that session closes

Formats are **verified** in the `pydeephaven` 42.6 source. The export lifetime comes from its docstrings and the shared-ticket lifetime from a test; scope and application ticket lifetimes are our understanding, not tested. A shared ticket's lifetime is the surprise: when its publisher closed, fetching the ticket failed, and so did a copy that another session had fetched while it was valid. To hand a table to other sessions durably, bind it to a name instead (`bind_table`), which makes it an ordinary field.

A typed ticket is a ticket plus its object type (Table, Figure, `deephaven.ui.Element` , …); listing a server's fields returns typed tickets.

Exports explain the unnamed nodes an introspector finds. When a grid sorts a column, the client asks for `sort(trades)` under a new export ticket. That is a real, ticking node in the worker's update graph, owned by the session and named nowhere; closing the browser tab releases it.

What clients can and cannot do to a worker's graph:

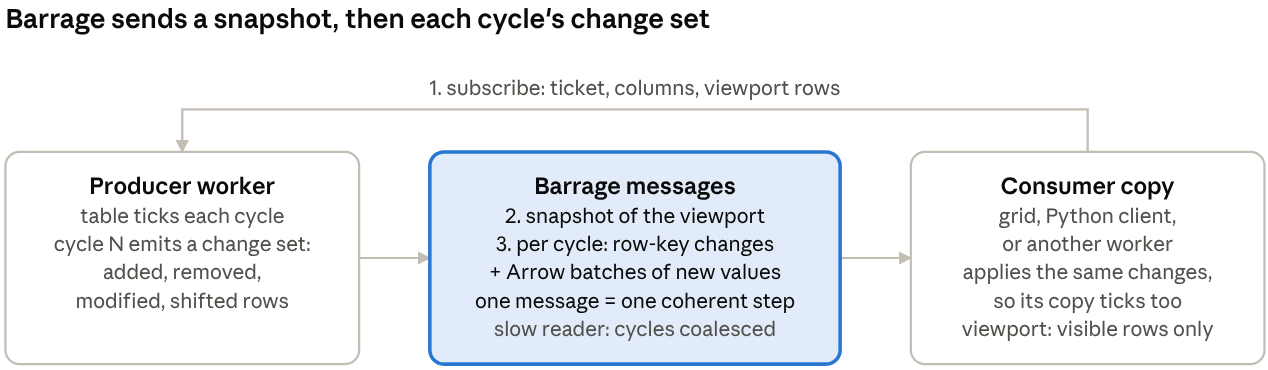
- **Branch from any node they hold a ticket to**, not only leaves: fields and their own exports, mid-graph or not.
- **Never change existing computation.** The graph only grows downstream; a parent computes exactly as before. The shared costs are more work per cycle, more memory, and a slow branch delaying the cycle for every table in the worker.

- **Add new roots**: upload a table, create an input table, or create a time table through the client API.
- **Mutate only through two doors**: writing data into an input table, or running code with `run_script` where console access is allowed.
- **Promote an export to a field** with `bind_table(name, table)`. It then outlives the session that created it. **Verified.**

Barrage: streaming ticking tables

Barrage extends Arrow Flight so a client receives a snapshot followed by incremental updates in the same shape the engine uses internally. Plain Flight can only send a table as a one-off batch.

Each update cycle a table emits a change set: rows added, removed, modified and shifted. Barrage serializes exactly that change set, plus Arrow record batches holding the new values. The consumer applies it to its own copy, which therefore ticks too.



Barrage subscription · request, snapshot, per-cycle change sets

The consumer never recomputes the producer's logic; it replays the producer's change sets onto its own copy.

- **Viewports.** A client subscribes to a row range and a set of columns, such as rows 0–49 of Sym and Px. The server sends only what is visible and changed, which is how a grid shows a million-row ticking table cheaply.
- **Consistency.** Each message reflects one coherent update-graph step on the producer, never half a cycle.
- **Coalescing.** A slow or throttled subscriber receives several cycles merged into one message, not a backlog.

- **Snapshots** are the simple case: fetch one static batch through a ticket without subscribing.

Where we already used it without naming it:

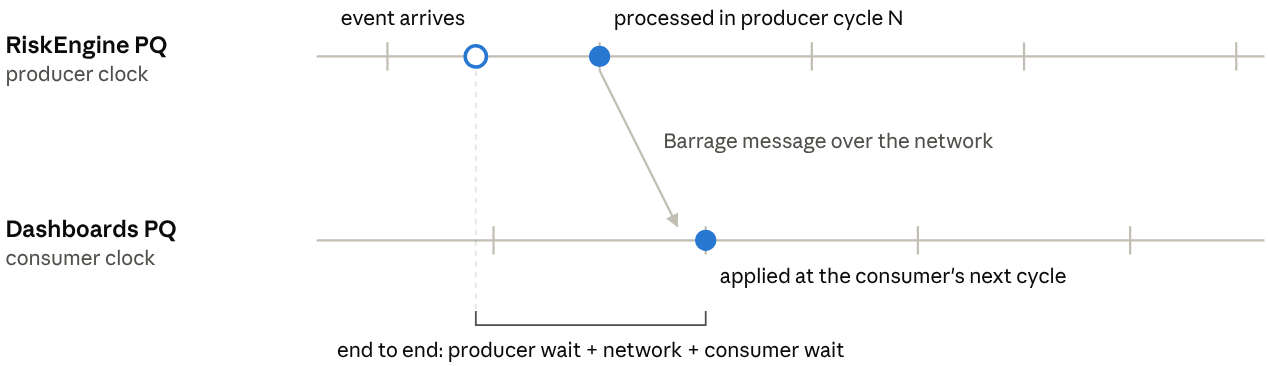
- Every grid in both UI prototypes is a Barrage viewport subscription; the React proxy rules for `/arrow.*` and `/io.deephaven.*` carry that traffic.
- The React app's live summary cards set a viewport and listened for updates.
- `to_arrow()` in our Python driver was a one-off snapshot.

Across workers on Enterprise

A table shared between PQs is a Barrage subscription, so it ticks on the consumer's clock, one network hop and one cycle behind the producer.

A remote table ticks on the consumer's clock, one hop behind

Each worker has its own update-graph clock; tick marks are cycles, spacing illustrative



two workers, two clocks · where cross-PQ latency comes from

A change waits for the producer's next cycle, crosses the network, then waits again for the consumer's next cycle.

- **Each PQ has its own global scope.** That is the `scope` in `pq://RiskEngine/scope/positions`: the field `positions` in the scope application of PQ RiskEngine. **Docs** (URI format).
- **The consumer gets a source node.** `uri.resolve("pq://...")` and `remote_table` return a Barrage-backed table, which is a source in the consumer's graph. **Docs** (data is copied into the consumer), **Unverified** (class naming).

- **Two clocks, not one.** Within one worker, a join between two tables is consistent at every step. Across workers, each side is consistent with itself, and the consumer sees the producer as of the last message it applied.
- **Latency adds up**: producer cycle, plus network, plus the consumer's next cycle.
- **Database tables live in no scope.** `db.live_table(...)` opens one into a scope by binding it to a name, at which point it becomes a field like any other. **Unverified.**
- **A Code Studio** is a temporary worker with its own, separate scope.

Identity, fields and persistence

An application id only means something inside the worker you are connected to; the durable identity on Enterprise is the PQ, and durable state lives only where the app writes it.

Level	Community	Enterprise
Persistent identity	None beyond the server address	The PQ: its name and serial, kept by the controller across restarts (**Docs**)
Which worker	Whatever runs at `host:port`	Whichever worker the controller started for that PQ this time
Namespace inside the worker	Application id: `scope` or an app-mode id	`scope`, as in `pq://RiskEngine/scope/positions`
Object	Field name	Field name

- **No API keys on application id across workers.** Every address names the server or PQ first; the app id never stands alone. **Verified** for URIs (`dh+plain://host:port/app/<id>/field/<name>`).
- **The same app id on two workers means nothing shared.** Each worker has its own registry. Across workers, fields are read-only: a subscriber gets a Barrage replica, and writes to a replica do not reach the original. The only ways to change another worker are writing to its input tables or running code in it, both as a client of that specific worker.
- **PQ redundancy** can run several workers for one PQ; as we understand it, each computes independently rather than co-owning fields. **Unverified.**

- **Fields are an address book, not a permission boundary.** They are the published handles clients build on, but anything a client may do beyond them is governed by authentication, console access and, on Enterprise, PQ permissions and table access rules.
- **Coordination between PQs is by convention.** Nothing checks that `RiskEngine` still publishes `positions` with the expected columns until a subscription fails.
- **Nothing in a worker persists.** When it stops, its fields are gone, and so is any record of what they were. What survives is the PQ definition in the controller and whatever the app wrote out: database intraday and historical tables, Kafka topics, files. A new worker rebuilds from those in its startup script.
- **No stored interface contract.** You cannot list a down PQ's fields; `remote_table` even fails against a stopped PQ unless the consumer supplies the table definition itself (**Docs**). If the interface matters, keep a versioned list of field names and types in source control and check it against the live worker, for example with the introspection report.

What fields add beyond any app plus a database is sharing a live table by reference: every client in a worker sees the same consistent step, and Barrage keeps other workers' replicas current incrementally.

What can be introspected after the fact

Most of an app can be inventoried from the running worker alone, without its source; the gap is dependency edges for nodes created before the ancestors log was opened.

What	Without the source?	Where it comes from
Exposed fields and their types	Yes	The server's field registry (all applications, including `scope`)
Named objects in scope, classified	Yes	The global scope
Graph nodes: operation and cost	Yes, back to worker start	Update performance log
Graph edges (what feeds what)	Only for nodes created after the ancestors log was first opened	Update performance ancestors log

What	Without the source?	Where it comes from
Unnamed nodes (client exports, leaks)	As complete as the edges	Both logs
Listeners: count, cost, description, stacked duplicates	Yes; the description is whatever the author passed	Update performance log
Threads, listening ports, loaded libraries	Yes	Python and JVM runtime, `/proc`
What a listener does, such as where it writes	No: the Python callable is held inside the engine's listener adapter	—
Functions in scope	Partly: signature, names used, file and line; no source for code typed into a console	Python function objects

The one request for app authors: open `perfmon.update_performance_ancestors_log()` at the very top of the startup script.

Our `introspect/dh_introspect.py` module and its client implement this table, and now run end to end (`tests/probe_introspection.py`). Against a worker running experiments 2–4, with a client holding a sorted export, the report found:

- all scope objects by kind, plus both probe apps' fields from the server registry;
- 2 Python listeners matching 2 handles in scope, using 34% of engine time;
- the three Python-fed stream tables, the timers, and two Barrage sources left by a URI test;
- 8 active nodes unreachable from any name, including the held client export;
- the `ex4-asyncio` thread as the only non-engine Python thread.

The first run also showed what to classify carefully. `merged_listen` creates one internal "recorder" listener per watched table, which looked like stacked duplicates; joins and `update_by` create internal helper nodes; and Java threads that call into Python appear in Python's thread list. Listening ports and process memory are read from `/proc`, so they are unavailable on Windows.

Confidence and sources

Verified means observed on Deephaven Community 42.6 or read in its client source; **Docs** means stated in Deephaven documentation; **Unverified** means our understanding, to confirm with the vendor. Unmarked statements are general background on how the pieces fit.

- Application mode
- Use the JS API
- Enterprise remote tables and URIs (Python)
- Enterprise PQ overview
- Companion doc: `Deephaven as an Application Framework: Findings and Guidance`

Tests behind the **Verified** marks: `tests/probe_concepts.py` with the probe apps in `tests/probe_app.d`, and `tests/probe_introspection.py`, plus tests/embedded_thread_demo.py and tests/probe_event_driven_clock.py (see the ledger VERIFIED_FACTS.md), run on Deephaven Community 42.6 (pip `deephaven-server`, native Windows, Java 17). Ticket formats come from the `pydeephaven` 42.6 source.